

Desarrollo web en entorno cliente

UD – 4

Promesas

Contenidos













ÍNDICE

Introducción **3**

Promesas **4**

Consumir promesas **4**

Método then *5*

Método catch *5*

Encadenando promesas *5*

1. INTRODUCCIÓN

La **asincronía** es uno de los conceptos principales que existe en el mundo de Javascript. Cuando comenzamos, es muy común trabajar con código síncrono (secuencial), sin embargo, llegado a un punto necesitaremos trabajar con código asíncrono, el cuál puede ser muy frustrante si no lo entendemos bien.

¿Qué es la sincronía?

En programación, cuando hablamos de sincronía, nos referimos a una **forma de ejecutar el código**. Cuando comenzamos a programar, normalmente realizamos tareas de forma síncrona, llevando a cabo tareas secuenciales que se ejecutan una detrás de otra, de modo que el orden o flujo del programa es sencillo y fácil de observar en el código:

```
primera_funcion();      // Tarea 1: Se ejecuta primero
segunda_funcion();      // Tarea 2: Se ejecuta cuando termina primera_funcion()
tercera_funcion();      // Tarea 3: Se ejecuta cuando termina segunda_funcion()
```

Sin embargo, en el mundo de la programación, tarde o temprano necesitaremos realizar operaciones asíncronas, especialmente en ciertos lenguajes como Javascript, donde tenemos que **realizar tareas que tienen que esperar a que ocurra un determinado suceso**. Este suceso tiene varias características:

Generalmente, no depende de nosotros
No solemos saber cuando va a ocurrir
A veces, ni siquiera sabemos si va a ocurrir



Nuestro código tiene que saber gestionar esas características, y reaccionar sólo cuando dicho suceso ocurra. sería muy interesante que nuestro código *«se quede a la espera»* mientras ejecutamos y realizamos otras tareas.

¿Ejemplos de tareas asíncronas?

En Javascript no todas las tareas son asíncronas, pero hay ciertas tareas que si lo son, y probablemente se entiendan mejor con ejemplos reales:

- ✔ Descargarse un archivo .json de otra web.
- ✔ Reproducir un .mp3 desde otra web.
- ✔ Un botón o campo de datos donde el usuario debe activar un mecanismo.
- ✔ Una orden al sintetizador de voz del navegador para que lea un mensaje.
- ✔ Una comunicación con un sensor del smartphone.

¿Cómo gestionar la asincronía?

Debemos aprender mecanismos para dejar claro en nuestro código Javascript, que ciertas tareas tienen que procesarse de forma asíncrona para quedarse a la espera, y otras deben ejecutarse de forma síncrona.

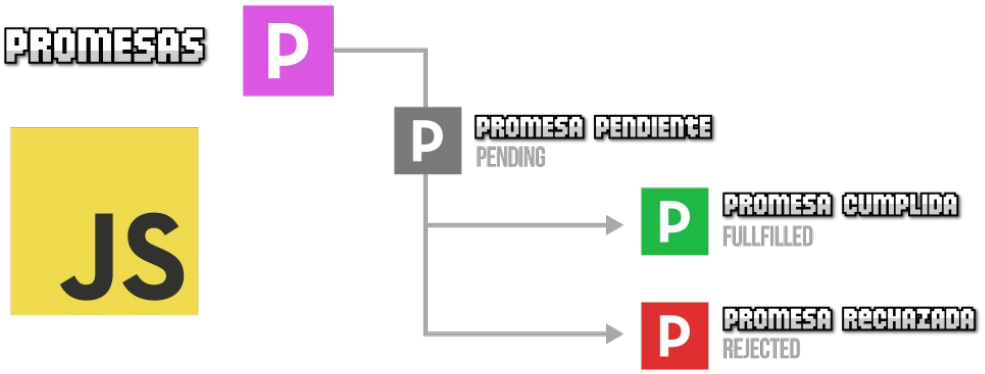
En Javascript existen varias formas de gestionar la asincronía. Observa en la siguiente tabla los principales mecanismos:

Mecanismo	Descripción	Más info
Mediante `callbacks`	Probablemente, la forma más clásica de gestionar la asincronía.	Ver Callbacks
Mediante `promesas`	Mecanismo moderno para gestionar la asincronía de forma no bloqueante.	Ver Promesas
Mediante `async/await`	Una forma simplificada de manejar promesas, pero bloqueante.	Ver Async/Await
Mediante `top-level await`	Una variación de la anterior, donde no es necesario usar `async` en determinados contextos.	

2. PROMESAS

Las **promesas** son un concepto para **simplificar el manejo de la asincronía** en Javascript de una forma mucho más elegante y práctica que, por ejemplo, la que se utilizaba antiguamente haciendo uso de las funciones **callbacks**, ya que era fácil que se volviese un código complejo y difícil de leer.

Como su propio nombre indica, en la programación al igual que en la vida real, una promesa es algo que en principio pensamos que se cumplirá en el futuro, pero a medida que pasa el tiempo pueden ocurrir varias cosas:



3. CONSUMIR PROMESAS

Las **promesas** en Javascript **son un tipo de dato** que puede ser utilizado de diferentes formas. Una de ellas es usar los **thennables**, donde se utiliza un método llamado .then(). Observa la siguiente tabla:

Métodos	Descripción	Más info
`.then( function resolve)`	Ejecuta la función cuando la promesa se cumple.	(Este artículo)
`.catch( function reject)`	Ejecuta la función cuando la promesa se rechaza.	(Este artículo)
`.then( function resolve, function reject)`	Equivale a las 2 anteriores en el mismo `.then()`.	Ver `then() + catch()`
`.finally( function end)`	Ejecuta la función cuando sale de pendiente.	Ver `finally()`

Recordemos que las promesas estarán siempre en un **estado** de los siguientes:

pendiente *(el estado inicial)*
aceptada *(se gestiona a través del .then())*
rechazada *(se gestiona a través del catch())*

3.1 Método then

vamos a utilizar la función asíncrona **fetch()**, que sirve para hacer peticiones y traernos archivos desde una dirección URL concreta. El método **then()** se ejecutará únicamente cuando la **promesa se cumpla.**

```
fetch("/robots.txt")
  .then(function(response) {
    // response: petición de respuesta del recurso solicitado
  })
```

3.2 Método catch

Es una buena práctica utilizar también el método **.catch()**, que al contrario que el método .then(), este método ejecutará la función callback pasada por parámetro cuando la **promesa se rechaza**:

```
fetch("/robots.txt")
  .then(function() {
    /* Código a realizar cuando se cumpla la promesa */
  })
  .catch(function() {
    /* Código a realizar cuando se rechaza la promesa */
  });
```

3.3 Encadenando promesas

En el caso del fetch(), si pedimos un archivo .json necesitaremos encadenar dos promesas, una para obtener la respuesta del servidor y otra para procesar el contenido JSON

```
fetch("/data.json")
  .then(response ⇒ {          // Gestiona la primera promesa
    return response.json();   // Devuelve otra promesa
  })
  .then(data ⇒ {              // Gestiona la segunda promesa
    console.log(data);        // Devuelve el JSON como un objeto ya procesado
  })
```